Teoria da Computação e Compiladores

Curso: Engenharia da Computação

Prof.: Renato Luciano Cagnin

Manual Técnico do Compilador de Missões de Drone

DSL formal em C# e WPF — teoria, arquitetura, implementação e uso documento de referência do projeto Compilador Drone

Linguagem de domínio para planejamento e mapeamento de missões agrícolas

Caroline da Silva Grizante Ra:114105

Emilly Emanuelly R. dos Santos Ra:114095

Marcela Lovatto Ra:113926

Fundação Hermínio Ometto

Araras-SP

Sumário

1.0 Como utilizar este manual..... pág.02
1.1 Visão rápida do pipeline..... pág.03
2.0 Objetivo e arquitetura do projeto..... pág.03
2.1 Princípio de responsabilidade..... pág.04
2.2 Organização das pastas..... pág.04
3.0 Fundamentos formais..... pág.05
3.1 Alfabeto da linguagem..... pág.05
3.2 Linguagem formal..... pág.05
4.0 Tokens, expressões regulares e normalização..... pág.06
4.1 Categorias de tokens..... pág.06
4.2 O objeto Token..... pág.06
4.3 Sinônimos..... pág.07
5.0 Lexer e autômato finito determinístico..... pág.08
5.1 Ideia do AFD..... pág.08
5.2 Leitura de uma linha, caractere por caractere..... pág.08
5.3 Números..... pág.08
5.4 Erro léxico..... pág.09
6.0 Gramática formal e Parser LL(1)..... pág.09
6.1 Gramática EBNF..... pág.09
6.2 Por que é LL(1)?..... pág.10
6.3 Funções do Parser..... pág.10
6.4 Derivação de exemplo..... pág.10
6.5 Erro sintático..... pág.10
6.6 Recuperação de erro..... pág.11
7.0 Modelos internos..... pág.11
7.1 TokenType e Token..... pág.11
7.2 Comando Missão..... pág.11
7.3 Missão..... pág.11
7.4 Fazenda..... pág.11
7.5 Símbolo..... pág.12
7.6 Diagnóstico e Relatório Missão..... pág.12
8.0 Análise semântica..... pág.12
8.1 Unidades obrigatórias..... pág.12
8.2 Valores positivos e faixas..... pág.13
8.3 Parâmetros duplicados..... pág.13
8.4 Localização e GeoGIS..... pág.14
8.5 Obstáculos e anticolisão..... pág.14
8.6 Meteorologia e ambiente..... pág.14
8.8 Rota, Home Point e operação..... pág.15
8.9 Regulamentação..... pág.15
9.0 Cobertura, consumo e memória..... pág.15

9.1 Cobertura com sobreposição..... pág.15
9.2 Consumo estimado..... pág.15
9.3 Memória requerida..... pág.16
9.4 Indicador de risco..... pág.16
10.0 Fluxo completo de execução..... pág.16
10.1 Inicialização da WPF..... pág.16
10.2 Execução do pipeline..... pág.16
10.3 O que acontece quando não há base geográfica..... pág.17
10.4 Atualização visual..... pág.17
11.0 Interface WPF..... pág.17
12.0 Diagnósticos e comportamento por fase..... pág.18
12.1 Ordem de interrupção..... pág.19
13.0 Exemplo completo acompanhado..... pág.19
13.1 Etapa lexical..... pág.20
13.2 Etapa sintática..... pág.20
13.3 Etapa semântica..... pág.20
13.4 Resultado visual..... pág.20
14.0 Casos de teste do projeto..... pág.21
15.0 Como executar e apresentar o trabalho..... pág.22
15.1 No Visual Studio..... pág.22
16.0 Limitações e decisões conscientes..... pág.22
16.1 Perfil técnico dos drones e compatibilidade da missão..... pág.23
16.1.1 Modelo de dados Drone..... pág.23
16.1.2 Catálogo drones.json..... pág.23
16.1.3 Sintaxe de seleção e sinônimos..... pág.24
16.1.4 Regras semânticas de compatibilidade..... pág.24
16.1.5 Exemplo de incompatibilidade..... pág.25
16.1.6 Integração com o pipeline..... pág.25
16.1.7 Novos campos do painel..... pág.26
16.1.8 Exemplos e validação..... pág.26
16.1.9 Limitações e escopo..... pág.26
17.0 Conclusão..... pág.26
18 Apêndice A — Código essencial resumido..... pág.27
18.1 Est rutura do comando..... pág.27
18.2 Consulta geográfica..... pág.28
18.3 Classificação de risco..... pág.28
19.0 Apêndice B — Checklist de avaliação..... pág.29
20.0 Apêndice C — Glossário..... pág.30

1.0 Como utilizar este manual

Este documento explica o projeto do compilador de missões de drone de ponta a ponta. A intenção é que uma pessoa consiga compreender o projeto mesmo sem abrir o código-fonte: o que é a linguagem, quais símbolos ela aceita, como uma linha é transformada em tokens, como a gramática é reconhecida, como os significados são verificados, como os erros são classificados e como os resultados chegam à interface WPF.

O projeto é uma DSL (Domain-Specific Language), isto é, uma linguagem específica de domínio. Ela não tenta entender frases livres em português. A entrada deve obedecer à forma formal de atribuições definida neste manual.

Escopo acadêmico: As fórmulas de consumo, cobertura, memória e risco são um modelo didático determinístico para demonstrar análise semântica. Elas não substituem planejamento aeronáutico, avaliação meteorológica profissional, autorização regulatória ou sistema de segurança de voo.

1.1 Visão rápida do pipeline

Fase	Recebe	Produz
1. Léxica	Caracteres do arquivo TXT	Tokens e diagnósticos léxicos
2. Sintática	Sequência de tokens	Objeto Missao e diagnósticos sintáticos
3. Semântica	Missao + dados geográficos	Relatório, métricas, alertas e risco
4. Interface	Relatório + tokens + símbolos	Console, tabelas e painel WPF

2.0 Objetivo e arquitetura do projeto

O aplicativo recebe um arquivo de texto com uma missão de drone. A missão descreve local, altitude, velocidade, área, bateria, condições ambientais, sobreposição de imagens, sensor, sistema anticolisão e regulamentação. A arquitetura respeita a separação clássica de um compilador:

Arquivo TXT

↓

Alfabeto

↓

Lexemas

↓

Tokens

↓

Expressões regulares

↓

Autômato

↓

Gramática

↓

Parser

↓

Semântica

↓

Erros

↓

Cálculos

↓

Drones

↓

Relatório

2.1 Princípio de responsabilidade

- O Lexer reconhece formas léxicas. Ele não decide se uma altitude é segura.
- O Parser reconhece a estrutura. Ele não calcula autonomia.
- O analisador semântico verifica significado, unidades, domínio e regras de risco.
- A interface não interpreta a linguagem diretamente: apenas orquestra as fases e apresenta seus resultados.

2.2 Organização das pastas

Caminho	Responsabilidade
Dados/fazendas.json	Base com dez propriedades fictícias, coordenadas, relevo e obstáculos
Exemplos/	Arquivos TXT de sucesso, sinônimos, erros e risco
Modelos/Token.cs	Categorias de tokens e posição lexical
Modelos/Missao.cs	Comandos reconhecidos e fazendas
Modelos/Simbolo.cs	Linha da tabela de símbolos
Modelos/ RelatorioMissao.cs	Diagnósticos, métricas e nível de risco

Pipeline/Lexer.cs	Analisador léxico e normalização
Pipeline/Parser.cs	Parser descendente recursivo LL(1)
Pipeline/ SemanticAnalyzer.cs	Regras semânticas e fórmulas
Pipeline/ ServicoGeoGIS.cs	Consulta da base geográfica
MainWindow.xaml	Layout WPF
MainWindow.xaml.cs	Eventos, pipeline e atualização visual

3.0 Fundamentos formais

3.1 Alfabeto da linguagem

Um alfabeto é um conjunto finito de símbolos. Na DSL, chamamos o alfabeto de Σ. A implementação aceita explicitamente:

Sigma = { A-Z, a-z, 0-9, _, -, ., =, %, /, espaço, tab, CR, LF }

Na prática:

- Letras ASCII maiúsculas e minúsculas
- Dígitos de 0 a 9
- Sublinhado em identificadores, como ALTURA_VOO
- Hífen somente para iniciar número negativo, como -21.8200
- Ponto em números decimais
- = como operador de atribuição
- % como unidade
- / somente na unidade km/h
- Espaços, tabulações e quebras de linha como separadores

Um caractere fora de Σ, como @, # ou &, não é ignorado. Ele gera um diagnóstico léxico com linha e coluna.

Alfabeto não é token: O alfabeto contém caracteres individuais. Um token é uma categoria produzida depois que vários caracteres foram agrupados pelo Lexer.

3.2 Linguagem formal

A linguagem L é o conjunto de o conjunto de textos que obedecem ao cabeçalho, aos comandos e aos valores permitidos pela gramática. Um arquivo pode pertencer sintaticamente à linguagem e ainda ser semanticamente inválido.

Por exemplo:

```
DEFINIR MISSAO
ALTURA_VOO = 80 km/h
```

A construção possui forma sintática correta: parâmetro, =, número e unidade. Porém, semanticamente, ALTURA_VOO exige metros. Portanto:

- Lexicalmente: válida;
- Sintaticamente: válida;
- Semanticamente: inválida.

Essa distinção é uma das ideias centrais do projeto.

4.0 Tokens, expressões regulares e normalização

4.1 Categorias de tokens

Token	Regra ou classe	Exemplo
TK_DEFINIR_MISSAO	Combinação de DEFINIR/INICIAR com MISSAO/VIAGEM	INICIAR VIAGEM
TK_IDENTIFICADOR	[A-Za-z_][A-Za-z0-9_]*	FAZ_002
TK_NUMERO	-?[0-9]+(\\.[0-9]+)?	-21.8200
TK_UNIDADE	km/h, mAh, ha, m, %, C	25 km/h
TK_ATRIBUICAO	Caractere literal =	=
TK_STATUS	ATIVO, INATIVO, SIM, NAO, SISANT_OK, PENDENTE	SISANT_OK
TK_FIM_ARQUIVO	Marcador inserido pelo Lexer	fim do arquivo

4.2 O objeto Token

Cada token contém:

```
public sealed record Token(
TokenType Tipo,
 string Lexema,
 int Linha,
 int Coluna,
 string? LexemaNormalizado = null)
```

- Tipo: categoria, como TK_ALTITUDE ou TK_NUMERO
- Lexema: texto original lido no arquivo

- Linha e Coluna: posição inicial para diagnósticos
- LexemaNormalizado: forma interna usada pelo Parser e pela semântica

Exemplo para:

ALTURA_VOO = 80 m

Lexema	Tipo	Normalizado	Posição
ALTURA_VOO	TK_ALTITUDE	ALTITUDE	linha 1, coluna 1
=	TK_ATRIBUICAO	=	linha 1, coluna 12
80	TK_NUMERO	80	linha 1, coluna 14
m	TK_UNIDADE	m	linha 1, coluna 17

4.3 Sinônimos

O dicionário permite que várias palavras sejam aceitas, mas conduzidas ao mesmo token:

Token interno	Formas aceitas	Exemplo
TK_ALTITUDE	ALTITUDE, ALTURA, ALTURA_VOO	ALTURA = 80 m
TK_VENTO	VENTO, VELOCIDADE_VENTO, RAJADA	RAJADA = 15 km/h
TK_AREA	AREA, AREA_TERRENO, TAMANHO_FAZENDA	AREA_TERRENO = 10 ha
TK_BATERIA	BATERIA, CAPACIDADE_BATERIA, CARGA	CARGA = 20000 mAh
TK_SENSOR	SENSOR, CAMERA, TIPO_CAMERA	CAMERA = RGB
TK_REGULAMENTACAO	REGULAMENTACAO, SISANT, DECEA, LICENCA	LICENCA = SISANT_OK
TK_DESVIO_OBSTACULOS	DESVIO_OBSTACULOS, SISTEMA_OBSTACULOS, ANTI_COLISAO	ANTI_COLISAO = ATIVO
TK_NUVENS / TK_CHUVA	NUVENS, NEBULOSIDADE, CHUVA, PRECIPITACAO	CHUVA = 0 mm/h
TK_GSD	GSD, RESOLUCAO_GSD	GSD = 4 cm/pixel

TK_OPERACAO	OPERACAO	OPERACAO = VLOS

A normalização não transforma a DSL em linguagem natural. Ela apenas cria variantes controladas para os mesmos conceitos.

5.0 Lexer e autômato finito determinístico

5.1 Ideia do AFD

O Lexer pode ser modelado como um AFD que percorre a entrada da esquerda para a direita. Ele mantém um estado e escolhe uma transição conforme o caractere atual.

Estado	Significado	Transições principais	Saída
q0	Inicial	letra ou _ → q1; dígito ou - → q2; = → q4	início
q1	Identificador	letra, dígito ou _ → permanece q1	identificador ou palavra normalizada
q2	Número	dígito → permanece q2; . → q3	número inteiro ou início decimal
q3	Parte decimal	dígito → permanece q3	número decimal
q4	Atribuição	aceita =	TK_ATRIBUICAO
qE	Erro	símbolo fora de Σ	diagnóstico LEX001

5.2 Leitura de uma linha, caractere por caractere

Considere:

```
ALTITUDE = 80 m
```

O processamento é:

1. A inicia identificador, entrando em q1
2. L, T, I, T, U, D, E mantêm q1
3. O espaço encerra o identificador
4. O Lexer consulta o dicionário e classifica ALTITUDE como TK_ALTITUDE
5. O espaço é ignorado
6. = produz TK_ATRIBUICAO
7. 80 inicia q2 e produz TK_NUMERO
8. m é reconhecido como TK_UNIDADE
9. A quebra de linha é separador

5.3 Números

A rotina LerNumero aceita número inteiro, decimal e decimal negativo:

```
80 -> TK_NUMERO
90.5 -> TK_NUMERO
-21.8200 -> TK_NUMERO
```

Um ponto sem dígitos após ele não é tratado como decimal válido. Um caractere isolado chega à regra de símbolo inválido e gera erro.

5.4 Erro léxico

Entrada:

```
ALTITUDE = 80 m
```

Quando o Lexer encontra @:

- não é espaço
- não é =
- não é %
- não inicia número
- não é letra ou _
- não pertence ao alfabeto

Ele cria:

```
LEX001: O símbolo '@' não pertence ao alfabeto da DSL.
```

O Lexer avança um caractere para não entrar em loop infinito e continua a leitura. A interface, porém, interrompe o pipeline antes do Parser quando existem diagnósticos léxicos, pois não é seguro construir uma análise sintática sobre uma sequência incompleta.

6.0 Gramática formal e Parser LL(1)

6.1 Gramática EBNF

A gramática utilizada pelo Parser é:

```
<missao>            ::= TK_DEFINIR_MISSAO <comandos> TK_FIM_ARQUIVO
<comandos>          ::= <comando> <comandos> | ε
<comando>           ::= <parametro> TK_ATRIBUICAO <valor> <unidade_opcional>
<unidade_opcional> ::= TK_UNIDADE | ε
<parametro>         ::= TK_VENTO | TK_UMIDADE | TK_ILUMINACAO
                      | TK_ALTITUDE | TK_SOBREPOSICAO
                      | TK_VELOCIDADE | TK_AREA | TK_BATERIA
                      | TK_SENSOR | TK_REGULAMENTACAO
                      | TK_DESVIO_OBSTACULOS | TK_IDENTIFICADOR
<valor>             ::= TK_NUMERO | TK_IDENTIFICADOR | TK_STATUS
```

O símbolo ε significa uma produção vazia. Ele aparece porque a lista de comandos pode terminar logo após o cabeçalho.

6.2 Por que é LL(1)?

LL(1) significa:

- primeiro L: leitura da entrada da esquerda para a direita
- segundo L: construção da derivação mais à esquerda
- 1: decisão usando um token de lookahead

Ao entrar em <comando>, basta olhar o token atual para saber se ele pode iniciar um parâmetro. Depois de consumir o parâmetro, o próximo token deve ser obrigatoriamente TK_ATRIBUICAO.

6.3 Funções do Parser

- Analisar: verifica o cabeçalho e inicia <comandos>
- Comandos: repete comandos até TK_FIM_ARQUIVO
- Comando: reconhece parâmetro, =, valor e unidade opcional
- EhInicioDeParametro: implementa o conjunto FIRST de <parametro>
- EhInicioDeValor: aceita número, identificador ou status
- Sincronizar: avança até encontrar um novo início de parâmetro ou fim

6.4 Derivação de exemplo

Para:

```
DEFINIR MISSAO
ALTITUDE = 80 m
```

A derivação é:

```
<missao>
=> TK_DEFINIR_MISSAO <comandos> TK_FIM_ARQUIVO
=> TK_DEFINIR_MISSAO <comando> <comandos> TK_FIM_ARQUIVO
=> TK_DEFINIR_MISSAO TK_ALTITUDE = <valor> <unidade_opcional> ...
=> TK_DEFINIR_MISSAO TK_ALTITUDE = TK_NUMERO TK_UNIDADE ...
```

6.5 Erro sintático

Entrada:

```
ALTITUDE = 80 m
```

O Parser consome TK_ALTITUDE. O lookahead é TK_NUMERO, mas a produção exige TK_ATRIBUICAO. Ele gera: SYN003: Esperado token TK_ATRIBUICAO (=). O Parser não diz que a

altitude é insegura, não calcula bateria e não verifica a fazenda. Essas decisões pertencem à semântica.

6.6 Recuperação de erro

Após um erro, Sincronizar avança tokens até encontrar:

- um token que possa iniciar outro parâmetro ou
- o marcador de fim do arquivo

Isso permite identificar mais de um problema em um arquivo, evitando que um erro faça o Parser ficar preso no mesmo token.

7.0 Modelos internos

7.1 TokenType e Token

TokenType é o enum que centraliza as categorias da linguagem. Token é um registro imutável que carrega tipo, lexema, posição e normalização.

7.2 Comando Missão

Representa uma atribuição já reconhecida pelo Parser:

```
ParametroToken   // token à esquerda de '='
ValorToken       // token depois de '='
UnidadeToken     // opcional
```

Propriedades importantes:

- NomeOriginal: preserva o texto digitado
- NomeNormalizado: permite consultas sem depender do sinônimo
- Valor e Unidade: usados pela semântica
- Linha e Coluna: preservam a origem do comando

7.3 Missão

É a estrutura sintática da missão. Contém o cabeçalho e uma lista de comandos. A função ObterPrimeiro procura um parâmetro normalizado, por exemplo ALTITUDE, independentemente de o usuário ter escrito ALTURA.

7.4 Fazenda

Modela um registro do JSON:

```
Id
Nome
```

```
CulturaPredominante
LatitudeCentral
LongitudeCentral
AlturaMaiorObstaculoM
TipoObstaculo
Relevo
```

7.5 Símbolo

É a linha da tabela de símbolos. Ele relaciona parâmetro original, valor, unidade, token normalizado e posição. Essa tabela é uma evidência visual da passagem do texto para a representação interna.

7.6 Diagnóstico e Relatório Missão

Diagnóstico contém:

- fase: léxica, sintática, semântica ou aplicação
- severidade: informação, alerta, erro ou crítico
- código
- mensagem
- linha, coluna e parâmetro

O Relatório Missão reúne tokens, símbolos, diagnósticos, fazenda localizada, consumo, bateria, memória, cobertura e risco.

8.0 Análise semântica

A semântica começa somente depois de uma estrutura sintaticamente válida. Ela responde perguntas que a gramática não consegue responder:

- A unidade é compatível?
- O valor está em faixa válida?
- A fazenda existe?
- A altitude supera os obstáculos?
- O vento está acima do limite?
- A bateria suporta o consumo?
- A regulamentação está aprovada?

8.1 Unidades obrigatórias

Parâmetro	Unidade	Regra
ALTITUDE	m	Altura de voo em metros.
VELOCIDADE	km/h	Velocidade de voo.

VENTO	km/h	Velocidade do vento.
AREA	ha	Área do terreno.
BATERIA	mAh	Capacidade disponível.
RESERVA_BATERIA	mAh	Margem reservada para terreno.
UMIDADE	%	Faixa de 0 a 100.
ILUMINACAO	%	Faixa de 0 a 100.
NUVENS	%	Faixa de 0 a 100, acima de 70 gera alerta.
CHUVA	mm/h	Acima de zero gera crítico meteorológico.
GSD	cm/pixel	Acima de 10 gera alerta de resolução.
SOBREPOSICAO	%	Faixa de 0 a 100.
SOBREPOSICAO_FRONTAL	%	Abaixo de 70 gera alerta.
SOBREPOSICAO_LATERAL	%	Abaixo de 60 gera alerta.
RAIO_MISSAO	km	Estima ida e retorno.

Se uma unidade obrigatória estiver ausente, o código SEM003 é emitido. Se estiver presente, mas for incompatível, é emitido SEM004.

8.2 Valores positivos e faixas

Altitude, velocidade, área e bateria devem ser maiores que zero. Umidade, iluminação e sobreposição devem ficar entre 0% e 100%.

- Valor não numérico onde se espera número → SEM009
- Valor menor ou igual a zero → SEM007
- Valor fora de faixa → SEM008

8.3 Parâmetros duplicados

Se o mesmo parâmetro aparece mais de uma vez, SEM005 é gerado como alerta. O modelo utiliza o primeiro valor, tornando a regra determinística.

8.4 Localização e GeoGIS

ServicoGeoGIS carrega o JSON e oferece duas estratégias:

1. Se FAZENDA existir, procura o ID sem diferenciar maiúsculas e minúsculas
2. Se não houver ID, exige LATITUDE e LONGITUDE, calcula a distância haversine para todas as propriedades e escolhe a mais próxima

O raio máximo acadêmico é de 25 km. A distância haversine é calculada por:

$$d = 2\,R\,\arcsin\left(\sqrt{\sin^2(\Delta\varphi/2)^2 + \cos(\varphi_1)\,\cos(\varphi_2)\,\sin^2(\Delta\lambda/2)}\right)$$

onde R = 6371 km. Se nenhuma propriedade estiver no raio, o diagnóstico GEO001 é gerado. Quando a propriedade é encontrada, GEO002 informa nome, distância e obstáculo máximo.

8.5 Obstáculos e anticolisão

A regra central é:

```
se ALTITUDE <= altura_maior_obstaculo_m:
    se ANTI_COLISAO = ATIVO ou SIM:
        alerta GPS001
    caso contrário:
        erro crítico GPS002
```

Exemplo: FAZ_003 tem obstáculo máximo de 45 m. Uma altitude de 30 m com ANTI_COLISAO = INATIVO gera risco CRÍTICO.

8.6 Meteorologia e ambiente

Além do vento acima de 30 km/h (MET001), a versão ampliada avalia chuva, nuvens, umidade e iluminação. CHUVA > 0 gera o crítico MET002, pois o modelo suspende a operação. NUVENS > 70% gera MET003, umidade acima de 90% gera MET004 e iluminação abaixo de 30% gera MET005. Essas regras são didáticas: não consultam uma estação meteorológica nem uma previsão real.

8.7 Mapeamento, sensores e relevo

GSD é registrado em centímetros por pixel; acima de 10 cm/pixel gera MAP001. As sobreposições frontal e lateral são verificadas separadamente (MAP002 e MAP003). Velocidade acima de 35 km/h gera MAP004, indicando possível motion blur conforme a câmera e o tempo de exposição. O sensor pode ser RGB, MULTIESPECTRAL, HIPERESPECTRAL ou LIDAR.

O compilador registra o tipo e emite uma informação, mas não processa imagens, não calcula NDVI/NDRE e não produz nuvem de pontos LiDAR. Para fazendas com relevo montanhoso ou íngreme, TERRAIN_FOLLOWING = ATIVO evita SEG003. O parâmetro representa uma decisão de planejamento; não há modelo digital de elevação real.

8.8 Rota, Home Point e operação

RAIO_MISSAO estima uma rota simplificada de ida e retorno, calculada como 2 * raio. O tempo é obtido dividindo a distância estimada pela velocidade. HOME_LATITUDE e HOME_LONGITUDE precisam ser informados juntos. RTH aceita ATIVO, INATIVO, SIM ou NAO; ele documenta o retorno automático, mas não controla o drone. OPERACAO = VLOS é aceita sem alerta adicional. OPERACAO = BVLOS gera SEG002, pois exige autorização e avaliação específica.

8.9 Regulamentação

REGULAMENTACAO = SISANT_OK é o estado permitido. PENDENTE ou outro estado gera REG002. Se o campo não for informado, a semântica emite REG001 como informação de que a missão não foi certificada por esse campo. O programa não acessa DECEA, SISANT ou qualquer serviço externo.

9.0 Cobertura, consumo e memória

Esta seção é especialmente importante porque os resultados exibidos no painel vêm de fórmulas explícitas.

9.1 Cobertura com sobreposição

Se AREA = A e SOBREPOSICAO = S, então:
fator = 1 - S/100
areaPlanejada = A / fator

A fórmula aumenta o esforço estimado porque imagens parcialmente repetidas precisam ser capturadas para formar o mosaico. A eficiência exibida pelo código é:

eficiencia = 100 / (1 + S/100)
Com 20% de sobreposição:
eficiencia = 100 / 1.2 = 83.3%

Essa eficiência é um indicador didático de aproveitamento do esforço de imageamento. Não é bateria, não é probabilidade de sucesso e não é percentual de conclusão.

9.2 Consumo estimado

A fórmula implementada é:

```
fatorSobreposicao = 1 - SOBREPOSICAO / 100
areaPlanejada = AREA / fatorSobreposicao
consumoBase = 300 + areaPlanejada * 250
fatorAltitude = 1 + max(0, ALTITUDE - 50) / 500
fatorVelocidade = 1 + abs(VELOCIDADE - 25) / 100
fatorVento = 1 + max(0, VENTO - 10) / 100
consumo = consumoBase * fatorAltitude * fatorVelocidade * fatorVento
```

Quando há obstáculo e anticolisão ativa, o código soma 500 mAh para representar o desvio planejado:

```
se anticolisão ativa e altitude <= obstáculo:
    consumo = consumo + 500
```

Se consumo > BATERIA, o diagnóstico AUT003 é emitido. Esse diagnóstico é alerta, mas o risco geral passa para ALTO.

9.3 Memória requerida

A estimativa didática é:

```
memoria = 128 + 8 * areaPlanejada + sensor
onde sensor vale 64 MB quando há sensor/câmera informado e 0 caso contrário.
```

A memória representa o armazenamento/processamento estimado das imagens. Não é memória RAM medida do computador.

9.4 Indicador de risco

O risco é uma classificação baseada em regras:

Nível	Condição
CRÍTICO	Existe ao menos um diagnóstico crítico, como colisão com anticolisão inativa.
ALTO	Existe erro semântico, autonomia insuficiente ou vento acima do limite.
MÉDIO	Existem alertas, mas não há erro nem crítico.
BAIXO	Não existem erros ou alertas semânticos relevantes.

Não é uma probabilidade estatística. O sistema não afirma que a missão tem, por exemplo, 80% de chance de sucesso.

10.0 Fluxo completo de execução

10.1 Inicialização da WPF

MainWindow carrega inicialmente Exemplos/missao_sucesso.txt, caso o arquivo exista. Os botões carregam outros exemplos. O botão Abrir TXT usa OpenFileDialog.

10.2 Execução do pipeline

A função ExecutarPipeline executa estas etapas:

1. cria Lexer
2. chama Analisar(EditorTextBox.Text)

3. guarda tokens
4. se houver erro léxico, atualiza a tela e para
5. cria Parser
6. chama Analisar(tokens)
7. se houver erro sintático, mostra os símbolos parciais e para
8. localiza Dados/fazendas.json
9. cria ServicoGeoGIS
10. cria SemanticAnalyzer
11. chama Analisar(missao, tokens)
12. atualiza console, abas, tabelas e indicadores

10.3 O que acontece quando não há base geográfica

A aplicação captura exceções de infraestrutura. O relatório recebe APP001 e o console informa a falha ao executar o pipeline. Nas versões corrigidas, os símbolos reconhecidos continuam sendo preservados antes da tentativa de carregar o JSON. O projeto também copia Dados/fazendas.json para a pasta de saída e procura o arquivo em diretórios-pai.

10.4 Atualização visual

A interface recebe o relatório e:

- preenche a tabela resumida de símbolos
- preenche a lista completa inferior de símbolos
- imprime cada token
- separa diagnósticos de informação dos alertas e erros
- atualiza risco, fazenda, obstáculo, consumo, bateria, cobertura e memória

11.0 Interface WPF

A janela é dividida em três faixas principais:

1. faixa superior: editor de missão e console
2. faixa central: símbolos resumidos e painel de análise
3. faixa inferior: cada token e cada símbolo

O painel de análise usa ScrollViewer, evitando que métricas como memória requerida sejam cortadas em janelas menores. O console usa abas:

- Resumo
- Informações
- Alertas e erros

Isso evita misturar uma mensagem positiva como GEO002 com um alerta de autonomia ou um erro de unidade.

12.0 Diagnósticos e comportamento por fase

Código	Fase	Significado
LEX001	Léxica	Caractere fora do alfabeto.
SYN001	Sintática	Cabeçalho não reconhecido.
SYN002	Sintática	Token não inicia comando nem é fim de arquivo.
SYN003	Sintática	Faltou = depois do parâmetro.
SYN004	Sintática	Faltou valor depois de =.
SEM001	Semântica	Parâmetro aceito pela gramática, mas não previsto no domínio.
SEM003	Semântica	Unidade obrigatória ausente.
SEM004	Semântica	Unidade incompatível.
SEM005	Semântica	Parâmetro duplicado.
SEM007	Semântica	Valor positivo informado como zero ou negativo.
SEM008	Semântica	Valor fora de faixa.
SEM009	Semântica	Valor numérico inválido.
GEO001	Semântica	Fazenda/coordenadas não localizadas.
GEO002	Informação	Propriedade localizada.
GPS001	Alerta	Altitude não supera obstáculo, mas anticolisão está ativa.
GPS002	Crítico	Risco de colisão com anticolisão inativa ou ausente.
MET001	Alerta	Vento acima de 30 km/h.
REG001	Informação	Regulamentação não informada.
REG002	Semântica	Regulamentação não permitida.
AUT001	Informação	Área não informada.
AUT003	Alerta	Consumo supera bateria.
AUT004	Informação	Autonomia compatível.

APP001	Aplicação	Falha de infraestrutura, como arquivo JSON ausente.

12.1 Ordem de interrupção

- erro léxico: interrompe antes do Parser
- erro sintático: interrompe antes da semântica, embora possa mostrar comandos parciais
- erro semântico: não interrompe necessariamente todas as regras; o relatório pode reunir vários diagnósticos
- alerta: não interrompe o pipeline
- informação: documenta uma conclusão positiva ou uma ausência não crítica

13.0 Exemplo completo acompanhado

Considere:

```
DEFINIR MISSAO
FAZENDA = FAZ_003
ALTITUDE = 80 m
VELOCIDADE = 25 km/h
AREA = 20 ha
BATERIA = 30000 mAh
RESERVA_BATERIA = 5000 mAh
VENTO = 18 km/h
UMIDADE = 92 %
ILUMINACAO = 65 %
NUVENS = 75 %
CHUVA = 0 mm/h
GSD = 3 cm/pixel
SOBREPOSICAO_FRONTAL = 80 %
SOBREPOSICAO_LATERAL = 70 %
SENSOR = MULTIESPECTRAL
TERRAIN_FOLLOWING = ATIVO
RAIO_MISSAO = 2 km
HOME_LATITUDE = -21.9100
HOME_LONGITUDE = -47.5200
RTH = ATIVO
OPERACAO = VLOS
ANTI_COLISAO = ATIVO
```

```
REGULAMENTACAO = SISANT_OK
```

13.1 Etapa lexical

O Lexer produz, entre outros:

```
TK_DEFINIR_MISSAO DEFINIR MISSAO
TK_IDENTIFICADOR FAZ_003
TK_ALTITUDE ALTITUDE
TK_ATRIBUICAO =
TK_NUMERO 80
TK_UNIDADE m
TK_VELOCIDADE VELOCIDADE
...
```

Nenhum caractere é inválido.

13.2 Etapa sintática

O Parser reconhece 10 comandos. Cada comando tem parâmetro, atribuição, valor e, quando necessário, unidade. A lista é convertida em Missao.Comandos.

13.3 Etapa semântica

- FAZ_003 existe e possui relevo montanhoso
- altitude de 80 m supera o obstáculo máximo de 45 m
- unidades, GSD e sobreposições são reconhecidos
- umidade de 92% e nuvens de 75% geram alertas
- Terrain Following ativo evita alerta de relevo
- sensor multiespectral é registrado sem calcular NDVI
- raio de 2 km produz rota de 4 km e tempo estimado
- SISANT_OK é permitido
- consumo é comparado com 30000 mAh
- risco final é MÉDIO por causa dos alertas meteorológicos

13.4 Resultado visual

O usuário vê:

- resumo do pipeline
- informação de localização na aba Informações
- informação de autonomia compatível, se aplicável
- tokens completos na área inferior

- símbolos completos na tabela inferior
- métricas no painel de análise

14.0 Casos de teste do projeto

Arquivo	Categoria	O que demonstra
01_missao_sucesso_completa.txt	Sucesso	Os principais parâmetros, coordenadas, clima, sensor, bateria e regulamentação.
02_missao_sucesso_mapeamento.txt	Sucesso	Sinônimos de mapeamento e câmera.
03_missao_sinonimos_completa.txt	Sinônimos	Normalização de altitude, bateria, overlap, câmera e licença.
04_erro_lexico_caractere.txt	Léxico	Símbolo @ fora de Σ.
05_erro_sintatico_atribuicao.txt	Sintático	Ausência do operador =.
06_erro_semantico_unidade.txt	Semântico	Altitude em km/h.
07_alerta_meteorologico_vento.txt	Meteorologia	Vento acima do limite.
08_risco_colisao_critico.txt	Segurança	Obstáculo acima da altitude e anticolisão inativa.
09_alerta_autonomia.txt	Autonomia	Área grande e bateria insuficiente.
10_erro_coordenadas_regulamentacao.txt	Geo/regulamento	Localização por coordenadas, sensor LiDAR e licença pendente.
11_missao_completa_fatores.txt	Versão ampliada	Chuva zero, GSD, dois overlaps, sensor multiespectral, relevo, RTH e VLOS.
12_alerta_chuva_bvlos.txt	Alertas ampliados	Chuva, nuvens, GSD alto, motion blur, BVLOS e rota sem RTH ativo.
missao_sucesso.txt	Sucesso	Exemplo carregado pelo botão principal da interface

missao_sinonimos.txt	Sinônimos	Normalização das formas alternativas da versão ampliada
missao_erro_lexico.txt	Léxico	Variação compacta do erro com @.
missao_erro_sintatico.txt	Sintático	Variação compacta da ausência de =.
missao_risco.txt	Risco	Variação compacta do risco de colisão.

15.0 Como executar e apresentar o trabalho

15.1 No Visual Studio

1. Abra CompiladorDrone.sln.
2. Use Windows com o workload Desktop development with .NET.
3. Confirme que Dados/fazendas.json e Exemplos/ estão sendo copiados para a saída.
4. Execute o projeto WPF.
5. Carregue um exemplo ou abra um TXT.
6. Explique primeiro o Lexer, depois o Parser e só então a semântica.

16.0 Limitações e decisões conscientes

- A DSL não possui strings entre aspas; valores textuais simples são identificadores.
- O Lexer usa letras ASCII para manter o alfabeto explícito. Nomes de propriedades com acentos estão no JSON, não necessariamente como identificadores da DSL.
- O Parser aceita identificadores genéricos como parâmetros para demonstrar a diferença entre sintaxe e semântica. A semântica marca parâmetros desconhecidos com SEM001.
- O modelo de consumo não é um modelo físico certificado e a rota é uma aproximação de ida e retorno.
- GSD é validado como parâmetro informado, mas não é calculado a partir de câmera, lente e sensor.
- O sistema não processa RGB, multiespectral, hiperespectral ou LiDAR; apenas registra o sensor.
- Chuva, nuvens, VLOS/BVLOS, RTH e Terrain Following são regras acadêmicas, não controles físicos.
- O risco é categórico e baseado em regras, não estatístico.
- A proximidade geográfica usa uma base fictícia e raio didático de 25 km.
- A interface apresenta o diagnóstico; ela não substitui a separação das fases no código.

16.1 Perfil técnico dos drones e compatibilidade da missão

A versão ampliada do projeto passou a distinguir três fontes de informação: o ambiente externo da fazenda, as capacidades técnicas do equipamento e os requisitos da missão. Essa separação torna a análise semântica mais realista, pois uma missão pode ser adequada para uma propriedade e, ainda assim, incompatível com o modelo de drone escolhido.

Dados/fazendas.json → localização, relevo e obstáculos
Dados/drones.json → características técnicas do equipamento
arquivo .txt → missão, requisitos e condições ambientais

O arquivo de missão seleciona o equipamento pelo comando DRONE. O Parser continua aceitando a forma geral parâmetro = valor [unidade], enquanto o SemanticAnalyzer consulta o catálogo e compara os valores.

16.1.1 Modelo de dados Drone

A classe Drone representa o perfil técnico de um modelo, incluindo sensor, resolução, GSD mínimo, velocidade máxima, faixa de altitude, teto de voo, alcance, bateria, resistência ao vento, proteção contra umidade e suporte a recursos de segurança.

```
public sealed class Drone
{
        public string Id { get; set; } = string.Empty;
        public string Modelo { get; set; } = string.Empty;
        public string TipoSensor { get; set; } = string.Empty;
        public double GsdMinimoCmPorPixel { get; set; }
        public double VelocidadeMaximaKmh { get; set; }
        public double TetoVooM { get; set; }
        public double AlcanceKm { get; set; }
        public double CapacidadeBateriaMah { get; set; }
        public double ResistenciaVentoKmh { get; set; }
        public bool SistemaAntiColisao { get; set; }
        public bool TerrainFollowing { get; set; }
        public bool Rth { get; set; }
}
```

16.1.2 Catálogo drones.json

O catálogo contém três modelos didáticos. Os valores são fictícios e servem para demonstrar a análise de compatibilidade; não representam especificações reais de fabricantes.

ID	Sensor	Bateria	Velocidade máx.	Teto	Alcance	Vento
DRONE_AGRI_A1	MULTIESPECTRAL	30000 mAh	36 km/h	120 m	8 km	40 km/h
DRONE_AGRI_B2	RGB	20000 mAh	25 km/h	100 m	4 km	25 km/h
DRONE_LIDAR_C3	LIDAR	40000 mAh	30 km/h	80 m	6 km	35 km/h

O serviço ServicoDrones carrega esse JSON no início da análise semântica e localiza o modelo pelo identificador informado no TXT. Se o identificador não existir, o compilador produz DRN001.

16.1.3 Sintaxe de seleção e sinônimos

```
DEFINIR MISSAO
DRONE = DRONE_AGRI_A1
FAZENDA = FAZ_002
ALTITUDE = 80 m
VELOCIDADE = 25 km/h
AREA = 10 ha
BATERIA = 30000 mAh
RESERVA_BATERIA = 4000 mAh
VENTO = 15 km/h
UMIDADE = 60 %
CHUVA = 0 mm/h
SENSOR = MULTIESPECTRAL
GSD = 4 cm/pixel
RAIO_MISSAO = 2 km
RTH = ATIVO
ANTI_COLISAO = ATIVO
REGULAMENTACAO = SISANT_OK
```

O Lexer reconhece DRONE como TK_DRONE. Também aceita MODELO_DRONE e AERONAVE como sinônimos do parâmetro. O valor DRONE_AGRI_A1 continua sendo um identificador, posteriormente resolvido pelo catálogo.

16.1.4 Regras semânticas de compatibilidade

Regra	Comparação	Diagnóstico
Velocidade	VELOCIDADE ≤ VelocidadeMaximaKmh	DRN003
Altitude	ALTITUDE ≤ TetoVooM	DRN004
Faixa operacional	mínimo ≤ ALTITUDE ≤ máximo	DRN005
Vento	VENTO ≤ ResistenciaVentoKmh	DRN006
Umidade	UMIDADE ≤ ProtecaoUmidadePercentual	DRN007
Chuva	CHUVA > 0 exige proteção específica	DRN008
Sensor	SENSOR = TipoSensor	DRN009
GSD	GSD solicitado ≥ GsdMinimoCmPorPixel	DRN010
Alcance	2 × RAIO_MISSAO ≤ 70% do alcance	DRN011
Recursos	anticolisão, Terrain Following e RTH	DRN012–DRN014
Bateria	capacidade informada comparada à cadastrada	DRN015

A margem de 70% do alcance é uma decisão didática para não planejar uma rota consumindo o alcance nominal completo. A distância estimada continua sendo calculada como ida e retorno pelo raio da missão.

16.1.5 Exemplo de incompatibilidade

```
DRONE = DRONE_AGRI_B2
ALTITUDE = 120 m
VELOCIDADE = 35 km/h
VENTO = 30 km/h
SENSOR = MULTIESPECTRAL
GSD = 2 cm/pixel
TERRAIN_FOLLOWING = ATIVO
ANTI_COLISAO = ATIVO
```

Esse exemplo seleciona um drone RGB com teto de 100 m, velocidade máxima de 25 km/h, resistência ao vento de 25 km/h, sem Terrain Following e sem anticolisão. A análise gera incompatibilidades DRN003, DRN004, DRN006, DRN009, DRN010, DRN012 e DRN013. O risco tende a CRÍTICO quando existe condição crítica ou incompatibilidade grave.

16.1.6 Integração com o pipeline

Lexer → reconhece TK_DRONE
Parser → cria ComandoMissao para DRONE
ServicoDrones → consulta drones.json
SemanticAnalyzer → compara missão e equipamento
RelatorioMissao → armazena drone e diagnósticos
MainWindow → exibe drone e limites no painel

O projeto mantém a separação clássica de responsabilidades: o Lexer não decide se o drone é adequado; o Parser não calcula autonomia; o analisador semântico consulta o domínio; a interface apenas apresenta o resultado.

16.1.7 Novos campos do painel

Campo	Conteúdo exibido
Drone	ID, modelo e sensor selecionados
Velocidade máxima do drone	Limite cadastrado em km/h
Teto de voo do drone	Limite cadastrado em metros
Resistência ao vento	Limite cadastrado em km/h
Alcance seguro	70% do alcance nominal
Proteção de umidade	Percentual máximo cadastrado

16.1.8 Exemplos e validação

Os casos de teste foram atualizados para incluir DRONE e foi acrescentado o arquivo 13_incompatibilidade_drone.txt. A coleção contém 18 arquivos TXT e cobre sucesso, sinônimos, erro léxico, erro sintático, erro semântico, risco de colisão, autonomia, regulamentação, fatores meteorológicos e incompatibilidade técnica do equipamento.

Na validação realizada após a atualização, o pipeline apresentou FALHAS=0 para os 18 arquivos. Os casos de sucesso foram alinhados aos sensores, baterias e recursos dos drones cadastrados.

16.1.9 Limitações e escopo

Os modelos são fictícios e a análise não substitui manual do fabricante, avaliação meteorológica profissional, planejamento aeronáutico, autorização regulatória ou sistema de prevenção de acidentes. A finalidade é acadêmica: demonstrar como uma DSL combina análise léxica, sintática, semântica e uma base de dados de domínio.

17.0 Conclusão

O projeto demonstra, em uma aplicação única, os principais conceitos de Teoria da Computação e Compiladores:

- alfabeto e linguagem formal
- expressões regulares
- autômato finito determinístico
- tokens e normalização
- gramática BNF/EBNF
- Parser descendente recursivo LL(1)
- árvore/estrutura interna simplificada da missão
- tabela de símbolos
- análise semântica e compatibilidade de unidades
- diagnósticos por fase
- integração com dados externos
- interface gráfica para inspeção dos resultados
- análise ampliada de chuva, nuvens, GSD, sobreposição frontal/lateral, sensores, relevo, RTH e VLOS/ BVLOS

A ideia mais importante é que uma missão só é considerada adequadamente analisada quando passa por todas as camadas: o Lexer confirma que os caracteres formam tokens, o Parser confirma que os tokens formam comandos e a análise semântica confirma que esses comandos fazem sentido no domínio de drones

18 Apêndice A — Código essencial resumido

18.1 Estrutura do comando

```
private void Comando()
{
    var parametro = Atual;
    Avancar();

    if (Atual.Tipo != TokenType.TK_ATRIBUICAO)
    {
        Erro("SYN003", "Esperado token TK_ATRIBUICAO (=).", Atual, parametro);
        Sincronizar();
        return;
    }

    Avancar();
    var valor = Atual;
    Avancar();
```

```
    Token? unidade = null;
    if (Atual.Tipo == TokenType.TK_UNIDADE)
    {
        unidade = Atual;
        Avancar();
```

18.2 Consulta geográfica

```
var maisProxima = Fazendas
    .Select(f => new {
        Fazenda = f,
        Distancia = DistanciaHaversineKm(
            latitude, longitude,
            f.LatitudeCentral, f.LongitudeCentral)
    })
    .OrderBy(x => x.Distancia)
    .FirstOrDefault();
```

18.3 Classificação de risco

```
if (relatorio.TemCritico)
        risco = "CRÍTICO";
else if (temErro || autonomiaInsuficiente || ventoAlto)
        risco = "ALTO";
else if (temAlerta)
        risco = "MÉDIO";
else
        risco = "BAIXO";
```

19.0 Apêndice B — Checklist de avaliação

OK	Item a conferir	Onde aparece
□	Alfabeto Σ explicitamente definido	Seção 3.1
□	Tokens e expressões regulares	Seção 4
□	Dicionário de sinônimos	Seção 4.3
□	AFD do Lexer	Seção 5
□	Gramática EBNF	Seção 6.1
□	Parser LL(1)	Seção 6
□	Separação léxico/sintático/semântico	Seções 1 e 8
□	Unidades e limites	Seção 8.1–8.2
□	Geografia e obstáculos	Seções 8.4–8.5
□	Autonomia e cobertura	Seção 9
□	Regulamentação e risco	Seções 8.9 e 9.4
□	Interface WPF e diagnósticos	Seções 10–12
□	Casos de teste	Seção 14

20.0 Apêndice C — Glossário

Termo	Definição
Alfabeto	Conjunto de símbolos permitidos pela linguagem.
AFD	Autômato Finito Determinístico que reconhece padrões léxicos.
AST/estrutura	Representação interna construída após a análise sintática; neste projeto, uma Missao com comandos.
DSL	Linguagem específica de domínio.
EBNF	Forma estendida de Backus–Naur para descrever gramáticas.
Lexema	Sequência concreta de caracteres que originou um token.
Lexer	Analisador léxico.
Lookahead	Token observado para decidir a produção sintática.
Parser	Analisador sintático.
Semântica	Significado e validade de domínio de uma construção.
Token	Categoria produzida pelo Lexer.
Unidade	Grandeza associada ao valor, como m, ha ou mAh.